Stat 175: Homework 5

matt krasnow

Spring 2026

Exercise 1 (Citation Network). Study the code provided in this Google Colab Notebook, which explores the citeseer dataset, a citation dataset consisting of 3,327 vertices and 9,107 edges. Each vertex can belong to one of six categories, which is represented by a label. The primary task is node prediction.

(*a*) Train a logistic regression model on the embeddings and node features obtained from node2vec. Compare its performance (in F1 metric) to a logistic regression model trained on the original features. Analyze the results and discuss any differences in performance between the two models.

(*b*) Implement a method to use the embeddings for edge prediction and evaluate it.

Solution to Exercise 1

(a) Node classification using original features and Node2vec embeddings.
Let $X \in \mathbb{R}^{n\times p}$ denote the original node-feature matrix, let $Z \in \mathbb{R}^{n\times d}$ denote the Node2vec embedding matrix, and let y denote the node labels. To make the comparison unambiguous, I report three models:

Model 1: logistic regression on X, Model 2: logistic regression on Z, Model 3: logistic regression on $[X \mid Z]$.

X measures what the raw features can do on their own, Z measures what the graph embedding can do on its own, and $[X \mid Z]$ measures whether the two sources of information complement one another.

Code.

```
from sklearn.linearchar95model import LogisticRegression
from sklearn.metrics import f1char95score
from sklearn.modelchar95selection import trainchar95testchar95split
from sklearn.preprocessing import StandardScaler
import numpy as np

idx = np.arange(len(y))
idxchar95train, idxchar95test = trainchar95testchar95split(
    idx, testchar95size=0.3, randomchar95state=0, stratify=y
)

Xchar95train, Xchar95test = X[idxchar95train], X[idxchar95test]
Zchar95train, Zchar95test = Z[idxchar95train], Z[idxchar95test]
ychar95train, ychar95test = y[idxchar95train], y[idxchar95test]

XZchar95train = np.concatenate([Xchar95train, Zchar95train], axis=1)
XZchar95test = np.concatenate([Xchar95test, Zchar95test], axis=1)

scalerchar95X = StandardScaler(withchar95mean=False)
Xchar95traincharsc = scalerchar95X.fitchar95transform(Xchar95train)
Xchar95testcharsc = scalerchar95X.transform(Xchar95test)

scalerchar95Z = StandardScaler()
Zchar95traincharsc = scalerchar95Z.fitchar95transform(Zchar95train)
```

```
Zchar95testchar95s = scalerchar95Z.transform(Zchar95test)

scalerchar95XZ = StandardScaler(withchar95mean=False)
XZchar95trainchar95s = scalerchar95XZ.fitchar95transform(XZchar95train)
XZchar95testchar95s = scalerchar95XZ.transform(XZchar95test)

dfchar95X = LogisticRegression(maxchar95iter=5000)
dfchar95X.fit(Xchar95trainchar95s, ychar95train)
predchar95X = dfchar95X.predict(Xchar95testchar95s)
fchar95X = f1char95score(ychar95test, predchar95X, averagechar39macrochar39)

dfchar95Z = LogisticRegression(maxchar95iter=5000)
dfchar95Z.fit(Zchar95trainchar95s, ychar95train)
predchar95Z = dfchar95Z.predict(Zchar95testchar95s)
fchar95Z = f1char95score(ychar95test, predchar95Z, averagechar39macrochar39)

dfchar95XZ = LogisticRegression(maxchar95iter=5000)
dfchar95XZ.fit(XZchar95trainchar95s, ychar95train)
predchar95XZ = dfchar95XZ.predict(XZchar95testchar95s)
fchar95XZ = f1char95score(ychar95test, predchar95XZ, averagechar39macrochar39)
```

Results and discussion.

- The model trained on X (original node features, capturing *content information*) achieved macro-F1 $= 0.674$.
- The model trained on Z (Node2vec embeddings, capturing *graph-structural information*) achieved macro-F1 $= 0.515$.
- The model trained on $[X \mid Z]$ (combining *content plus network position*) achieved macro-F1 $= 0.679$.

The original node features substantially outperform the Node2vec embeddings alone. This indicates that for Citeseer the bag-of-words content features are already highly informative about paper category. The graph embeddings on their own capture some class signal through the citation structure, but not as much. The combined model $[X \mid Z]$ slightly improves over X alone (from 0.674 to 0.679), confirming that the graph embedding contributes a small amount of relational information not already present in the raw features.

(b) Edge prediction from Node2vec embeddings.
To use embeddings for edge prediction, we must avoid letting the embedding model see the test edges. The procedure is:

1. split the observed edges into training edges and test edges,
2. train Node2vec on the graph containing only the training edges,
3. create positive examples from held-out true edges,
4. create negative examples from non-edges,
5. convert each node pair (u, v) into an edge feature vector,
6. train a binary classifier on the training pairs and evaluate it on the test pairs.

A standard pairwise feature map is the Hadamard product

$$\varphi(u, v) = z_u \odot z_v.$$

This turns two node embeddings into a fixed-length feature vector for the edge.

Code.

```
import numpy as np
import networkx as nx
from node2vec import Node2Vec
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, average_precision_score
from sklearn.model_selection import train_test_split

edges = np.array(list(G.edges()))
num_nodes = G.number_of_nodes()

train_edges, test_edges = train_test_split(
    edges, test_size=0.3, random_state=0
)

G_train = nx.Graph()
G_train.add_nodes_from(G.nodes())
G_train.add_edges_from(train_edges)

node2vec = Node2Vec(
    G_train, dimensions=64, walk_length=20, num_walks=100,
    p=1, q=1, workers=1, seed=0
)
model = node2vec.fit(window=10, min_count=1, batch_words=4)

Z = np.vstack([model.wv[str(v)] for v in range(num_nodes)])

train_edge_set = set()
for u, v in train_edges:
    a, b = min(u, v), max(u, v)
    train_edge_set.add((a, b))

all_edge_set = set()
for u, v in edges:
    a, b = min(u, v), max(u, v)
    all_edge_set.add((a, b))

rng = np.random.default_rng(0)

def sample_negative_edges(num_samples, forbidden_edges, num_nodes, rng):
    neg_edges = []
    neg_set = set()
    while len(neg_edges) < num_samples:
        u = rng.integers(0, num_nodes)
        v = rng.integers(0, num_nodes)
        if u == v:
            continue
        a, b = min(u, v), max(u, v)
        if (a, b) in forbidden_edges or (a, b) in neg_set:
            continue
        neg_edges.append((a, b))
        neg_set.add((a, b))
    return np.array(neg_edges)

neg_train = sample_negative_edges(len(train_edges), all_edge_set, num_nodes,
    rng)
neg_test = sample_negative_edges(len(test_edges), all_edge_set, num_nodes, rng
    )
```

```
def hadamard_features(pairs, Z):
    return np.array([Z[u] * Z[v] for u, v in pairs])

X_pos_train = hadamard_features(train_edges, Z)
X_neg_train = hadamard_features(neg_train, Z)
X_pos_test = hadamard_features(test_edges, Z)
X_neg_test = hadamard_features(neg_test, Z)

X_train = np.vstack([X_pos_train, X_neg_train])
y_train = np.hstack([np.ones(len(X_pos_train)), np.zeros(len(X_neg_train))])

X_test = np.vstack([X_pos_test, X_neg_test])
y_test = np.hstack([np.ones(len(X_pos_test)), np.zeros(len(X_neg_test))])

clf = LogisticRegression(max_iter=5000)
clf.fit(X_train, y_train)

probs = clf.predict_proba(X_test)[:, 1]

auc = roc_auc_score(y_test, probs)
ap = average_precision_score(y_test, probs)
```

Results and discussion. I used the Hadamard product of the Node2vec embeddings as edge features and trained a logistic regression classifier to distinguish true citation edges from sampled non-edges. To avoid data leakage, the embeddings were trained only on the graph with the training edges, while evaluation was performed on held-out edges. The model achieved ROC-AUC = 0.665 and average precision = 0.764, indicating that the embedding preserves meaningful structural information for link prediction. ROC-AUC and average precision are natural metrics here since edge prediction is fundamentally a ranking problem; accuracy would depend on an arbitrary threshold.
This method works because Node2vec learns vectors that reflect random-walk co-occurrence patterns. Nodes in related graph neighborhoods receive similar embeddings, and adjacent nodes have interactions in embedding space that differ from those of random non-neighbors.

Exercise 2 (DeepWalk). DeepWalk is a predecessor to Node2vec that relies on standard, unbiased random walks (equivalent to Node2vec with $p = 1$ and $q = 1$). Let G be a complete bipartite graph $K_{m,n}$ with partition sets U and V.

(a) Calculate the stationary distribution of the DeepWalk random walk on $K_{m,n}$.

(b) In a standard DeepWalk, what is the probability that a walk of length $L \geq 2$ starting at $u \in U$ will sample another node $u' \in U$ at an odd time step (e.g., $t = 1, 3, 5$)?

(c) Suppose we want to generate embeddings that strongly cluster nodes of the same partition (U together, V together). How should you set p and q in Node2vec to encourage this structural equivalence over homophily? Justify your choice with a probability calculation.

Solution to Exercise 2

Let $G = K_{m,n}$ with bipartition $U \cup V$, where $|U| = m$ and $|V| = n$. Every vertex in U has degree n, and every vertex in V has degree m.

(a) Stationary distribution.
For a simple random walk on any finite undirected graph, the stationary distribution is proportional

to degree:

$$\pi(v) = \frac{\deg(v)}{\sum_{w \in V(G)} \deg(w)}.$$

Since the graph $K_{m,n}$ has mn edges, we have

$$\sum_{w \in V(G)} \deg(w) = 2|E| = 2mn.$$

Therefore, for any $u \in U$,

$$\pi(u) = \frac{n}{2mn} = \frac{1}{2m},$$

and for any $v \in V$,

$$\pi(v) = \frac{m}{2mn} = \frac{1}{2n}.$$

So the stationary distribution is uniform inside each partition. Summing over each side gives

$$\sum_{u \in U} \pi(u) = m \cdot \frac{1}{2m} = \frac{1}{2}, \qquad \sum_{v \in V} \pi(v) = n \cdot \frac{1}{2n} = \frac{1}{2}.$$

Hence, in stationarity, the walk spends half its time in U and half its time in V.

(b) Probability of sampling another node $u' \in U$ at an odd time step.
Because $K_{m,n}$ is bipartite, every step of the walk switches sides. If the walk starts at a node in U, then the parity of time completely determines the partition:

$$U \to V \to U \to V \to \cdots$$

Thus

- at even times $t = 0, 2, 4, \ldots$, the walk is in U,
- at odd times $t = 1, 3, 5, \ldots$, the walk is in V.

Therefore the probability that the walk is at another node $u' \in U$ at an odd time step is

$$\boxed{0.}$$

(c) Choosing p and q in Node2vec to favor structural equivalence.
Recall the Node2vec transition rule. Suppose the walk just moved from t to v, and now must choose the next node $x \in N(v)$. The unnormalized weight assigned to x is

$$\alpha_{pq}(t, x) = \begin{cases} 1/p & \text{if } d(t, x) = 0, \\ 1 & \text{if } d(t, x) = 1, \\ 1/q & \text{if } d(t, x) = 2. \end{cases}$$

Now suppose the walk has just moved

$$u \in U \to v \in V.$$

Every neighbor of v lies in U. Among these neighbors:

- the previous node u has distance $d(u, u) = 0$, so its weight is $1/p$,
- every other node $u' \in U$ with $u' \neq u$ has distance $d(u, u') = 2$, so each such node receives weight $1/q$.

Hence the total unnormalized mass is

$$\frac{1}{p} + (m-1)\frac{1}{q}.$$

So the probability of immediately returning to u is

$$\mathbb{P}(X_{k+1} = u \mid X_k = v,\ X_{k-1} = u) = \frac{\frac{1}{p}}{\frac{1}{p} + (m-1)\frac{1}{q}},$$

while the probability of moving to a particular different node $u' \in U$, $u' \neq u$, is

$$\mathbb{P}(X_{k+1} = u' \mid X_k = v,\ X_{k-1} = u) = \frac{\frac{1}{q}}{\frac{1}{p} + (m-1)\frac{1}{q}}.$$

If we want nodes in the same partition to appear in similar random-walk contexts, we want the walk to avoid immediate backtracking and instead spread across many different nodes on the same side of the bipartition over alternating steps. That means we should make returning to the previous node unlikely and transitions to other distance-2 nodes likely. Therefore we should choose

$$\boxed{p \text{ large and } q \text{ small.}}$$

Indeed, when p is large, the quantity $1/p$ is small, so backtracking is discouraged. When q is small, the quantity $1/q$ is large, so moving toward distance-2 nodes is encouraged. In the extreme regime $p \gg 1$ and $q \ll 1$,

$$\frac{\frac{1}{p}}{\frac{1}{p} + (m-1)\frac{1}{q}} \approx 0, \qquad \frac{\frac{1}{q}}{\frac{1}{p} + (m-1)\frac{1}{q}} \approx \frac{1}{m-1}.$$

So once the walk crosses to one side of the graph, it tends to spread out among many nodes in that partition rather than bouncing back to the same vertex. This makes vertices in the same partition appear in similar contexts, which is exactly the structural-equivalence behavior we want.

Exercise 3 (Fairness). In this exercise, we will explore fairness issues in Node2vec. Take a network $G = (V, E)$ with vertices having feature information $(x_v)_{v \in V}$ and protected attributes $(a_v)_{v \in V}$ (such as gender). Suppose that the protected attributes can take the value 0 or 1 and that they are equally likely. The response variables are given by

$$y_v = X_v^T \beta + \beta_2 a_v + \varepsilon_v,$$

and y_v is observed for a fraction of the nodes. In many cases one wants to make predictions that do not depend on the attributes.

(*a*) Simulate and visualize a graph from a Stochastic Block Model (SBM) where community assignments are strictly determined by the protected attributes (i.e., vertex v is in community 0 if $a_v = 0$, and community 1 if $a_v = 1$). Please use the intra-community and inter-community edge probabilities of $p_{0,0} = p_{1,1} = 0.1$ and $p_{1,0} = p_{0,1} = 0.01$.

(*b*) Compute the node representations $z_v \in \mathbb{R}^d$ using the standard Node2vec algorithm. Project these embeddings into two dimensions and plot them, coloring the vertices by a_v. What geometric structure do you observe?

(*c*) Train a regression model to predict y_v using the concatenated vectors (z_v, x_v). Evaluate the fairness of the resulting predictions. Are the predictions independent of a_v?

(*d*) Explain the mechanisms driving your observations in parts (*b*) and (*c*). Specifically, how does the random walk objective interact with the SBM transition probabilities to produce this outcome?

(e) Propose a specific modification to the random walk generation process that decorrelates the structural sampling from a_v to resolve this issue.

Solution to Exercise 3

(a) Simulating and visualizing the SBM.
We generate $n = 300$ nodes, assign each one a protected attribute $a_v \in \{0, 1\}$ independently with equal probability, and then define the community label to be exactly a_v. Edges are sampled according to

$$\mathbb{P}((u, v) \in E \mid a_u = a_v) = 0.1, \qquad \mathbb{P}((u, v) \in E \mid a_u \neq a_v) = 0.01.$$

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
n = 300
a = rng.integers(0, 2, size=n)

G = nx.Graph()
G.add_nodes_from(range(n))

for i in range(n):
    for j in range(i+1, n):
        p = 0.1 if a[i] == a[j] else 0.01
        if rng.random() < p:
            G.add_edge(i, j)

pos = nx.spring_layout(G, seed=0)
colors = ["tab:blue" if a[v] == 0 else "tab:orange" for v in G.nodes()]

plt.figure(figsize=(7, 7))
nx.draw_networkx_nodes(G, pos, node_color=colors, node_size=30, alpha=0.9)
nx.draw_networkx_edges(G, pos, alpha=0.15)
plt.axis("off")
plt.show()
```

The resulting graph has 300 nodes and 2497 edges. Because the within-group connection probability is ten times the across-group probability, the spring layout displays two clearly separated clusters, one for $a_v = 0$ (137 nodes) and one for $a_v = 1$ (163 nodes).

(b) Node2vec embeddings and their geometry.
We compute Node2vec embeddings $z_v \in \mathbb{R}^{32}$ and project them to two dimensions using PCA.

```
from node2vec import Node2Vec
from sklearn.decomposition import PCA

node2vec = Node2Vec(
    G, dimensions=32, walk_length=20, num_walks=100,
    p=1, q=1, workers=1, seed=0
)
model = node2vec.fit(window=10, min_count=1, batch_words=4)

Z = np.vstack([model.wv[str(v)] for v in range(n)])

Z2 = PCA(n_components=2).fit_transform(Z)
```

```
plt.figure(figsize=(7, 6))
for group, color in [(0, 'tab:blue'), (1, 'tab:orange')]:
    idx = np.where(a == group)[0]
    plt.scatter(Z2[idx, 0], Z2[idx, 1], s=20, alpha=0.8, label=f'a={group}',
        color=color)
plt.legend()
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

The projected embeddings form two completely separated clouds. One cluster corresponds to nodes with $a_v = 0$ and the other to nodes with $a_v = 1$. A logistic regression on just the 2D PCA projections achieves 100% accuracy at predicting a_v. The embedding geometry is strongly correlated with the protected attribute, because Node2vec places nodes close together when they share similar random-walk contexts, and in this SBM the random-walk context is almost entirely determined by community membership.

(c) Regression on (z_v, x_v) and fairness.
We generate $y_v = x_v^T \beta + \beta_2 a_v + \varepsilon_v$ with $\beta_2 = 1$ and fit a linear regression using (z_v, x_v).

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

p = 5
X = rng.normal(size=(n, p))
beta = rng.normal(size=p)
beta2 = 1.0
eps = rng.normal(scale=0.5, size=n)
y = X @ beta + beta2 * a + eps

idx = np.arange(n)
train_idx = rng.choice(idx, size=int(0.6 * n), replace=False)
test_idx = np.setdiff1d(idx, train_idx)

XZ = np.concatenate([X, Z], axis=1)

reg = LinearRegression()
reg.fit(XZ[train_idx], y[train_idx])
yhat = reg.predict(XZ[test_idx])

mse = mean_squared_error(y[test_idx], yhat)
corr = np.corrcoef(yhat, a[test_idx])[0, 1]
mean0 = yhat[a[test_idx] == 0].mean()
mean1 = yhat[a[test_idx] == 1].mean()
```

The predictions are *not* independent of a_v. Even though a_v is never explicitly provided as an input feature, the results show:

- Test MSE $= 0.310$,
- $\mathrm{Corr}(\hat{y}_v, a_v) = 0.419$,
- Mean prediction for $a_v = 0$: 0.175; for $a_v = 1$: 0.968.

The substantial correlation and the large gap in group means confirm that the embedding z_v acts as a proxy for a_v, allowing the regression to reconstruct the protected-attribute effect. The predictor is therefore not fair in the sense of independence from the protected attribute.

(d) Mechanism driving parts (b) and (c).

The key mechanism is the interaction between the SBM transition probabilities and the Node2vec random-walk objective. In the SBM, nodes are much more likely to connect to others in the same protected group than to nodes in the other group, because

$$p_{\text{within}} = 0.1 \qquad \text{and} \qquad p_{\text{across}} = 0.01.$$

As a result, a random walk started at a node in community 0 spends most of its time within community 0, and similarly for community 1. The co-occurrence statistics seen by the embedding method are therefore heavily community-specific.
Node2vec optimizes the embeddings so that nodes occurring in similar walk windows receive similar vectors. Since nodes with the same value of a_v have similar walk windows, the learned embeddings cluster by protected group. Once this happens, a downstream regression model can recover information about a_v from z_v, even though the protected attribute was never explicitly provided. The mechanism is summarized by the chain

$$a_v \longrightarrow \text{community assignment} \longrightarrow \text{random-walk neighborhoods} \longrightarrow z_v \longrightarrow \hat{y}_v.$$

(e) A modification to the random walk process to reduce the dependence on a_v.
A concrete fix is to use an *attribute-balanced random walk*. At each step, instead of choosing the next neighbor uniformly, we reweight the transitions so that neighbors with $a = 0$ and neighbors with $a = 1$ are sampled with equal total probability whenever both groups are present.
Let $N_0(v) = \{u \in N(v) : a_u = 0\}$ and $N_1(v) = \{u \in N(v) : a_u = 1\}$. Define the transition probabilities as:

- If both $N_0(v)$ and $N_1(v)$ are nonempty: assign probability $\frac{1}{2|N_0(v)|}$ to each $u \in N_0(v)$ and $\frac{1}{2|N_1(v)|}$ to each $u \in N_1(v)$.
- If only one protected group appears in $N(v)$: assign probability $\frac{1}{\deg(v)}$ to each neighbor.

This transition rule forces the random walk to sample neighborhoods that are not dominated by the local protected-group imbalance. The co-occurrence statistics driving Node2vec then become less predictive of a_v, and the embeddings become less entangled with the protected attribute.
Other related fixes could also work, such as rejection sampling to force more cross-group transitions, inverse-propensity weighting of transitions, or adversarial debiasing after the embedding step. But the most direct approach is to modify the walk itself so that the sampling kernel is balanced with respect to the protected attribute.